Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05

Advanced Collections and Error Handling

Intro

This week, I learned how to use dictionaries, JSON files, and exception handling in Python. This assignment was similar to Assignment 04, which used lists and CSV files to collect, display, and save student registration information. Assignment 05 required modification of a starter program that used CSV operations and lists to use dictionaries, JSON files, and error handling.

Creating the Program

Dictionaries

The first step was to review the starter code and compare it to the assignment requirements. The original program used lists to store student registrations in a CSV file to save the data. I changed "student_data" from a list to a dictionary by updating square brackets to curly brackets and expected data type from “list” to “dict”. This allowed each student's information to be stored using the keys "FirstName", "LastName", and "CourseName". Next, I imported the "json" module named "Enrollments.json" (Figure 1).

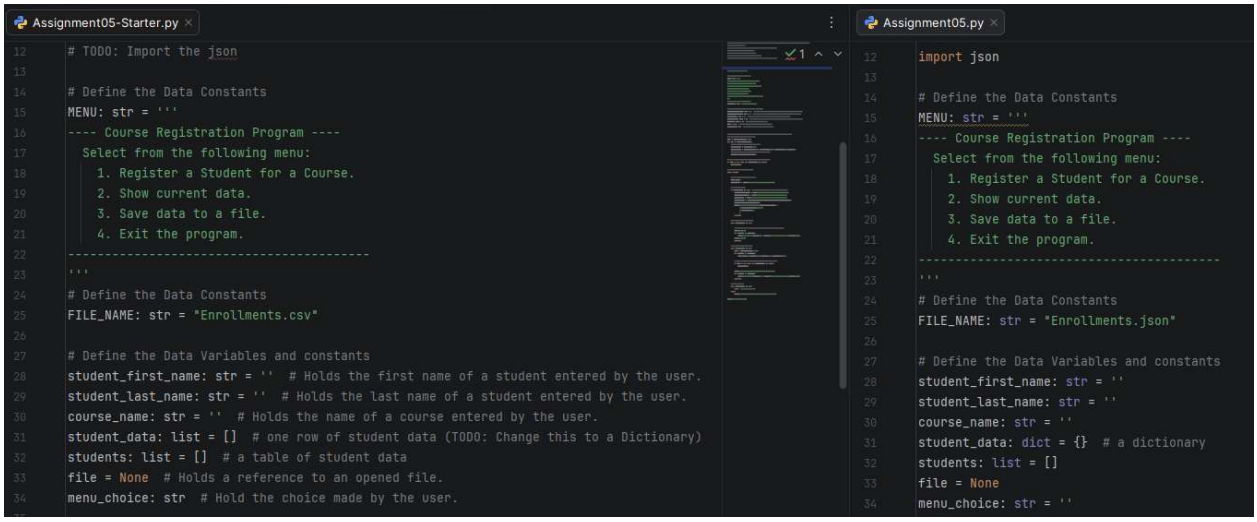


Figure 1: This image contains side-by-side sections of corresponding code (lines 12-34), of the starter code (left), and the modified code (right); please note the changes: importing the json (line 12), and updating the student list to a dictionary (line 31).

JSON and Error Handling

I replaced the original CSV operations for reading, appending, and saving data (Figure 2, Figure 3). “json.load()” was used to read the existing registrations (Figure 4). “append()” was moved to a while-loop protected section (Figure 5) to take user input in Menu Option 1. The dictionaries

Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05

were set up to display in Option 2 using the keys. The data was set up to be saved in Menu Option 3 with “json.dump()” (Figure 6). Error handling was included throughout the new program (Figure 4, Figure 5, Figure 6).

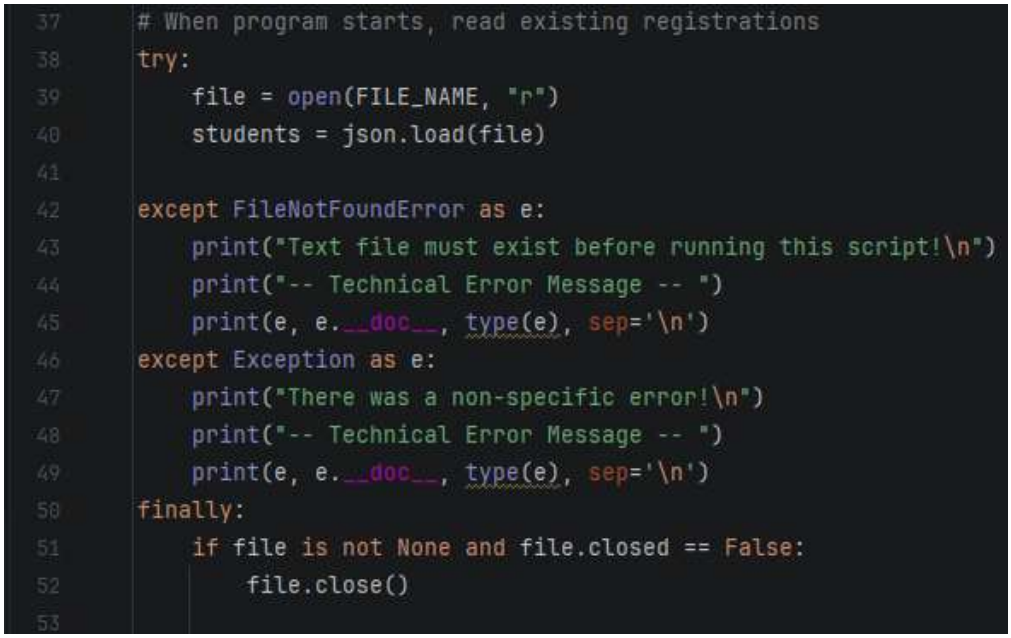
```
37  # When the program starts, read the file data into a list of lists (table)
38  # Extract the data from the file
39  file = open(FILE_NAME, "r")
40  for row in file.readlines():
41      # Transform the data from the file
42      student_data = row.split(',')
43      student_data = [student_data[0], student_data[1], student_data[2].strip()]
44      # Load it into our collection (list of lists)
45      students.append(student_data)
46
47  # Check if a file object exists and is still open
48  if file is not None and file.closed == False:
49      file.close()
```

Figure 2: This image contains the code to open, read, load into a collection, and close a CSV file taken from the starter program (lines 37-49). Please note append() function in line 45 and the lines 47-49, where the starter program checks for an existing file and closes it if it does exist.

```
81      # Save the data to a file
82      elif menu_choice == "3":
83          file = open(FILE_NAME, "w")
84          for student in students:
85              file.write(f"{student[0]},{student[1]},{student[2]}\n")
86
87          # Check if a file object exists and is still open
88          if file is not None and file.closed == False:
89              file.close()
90
91          print("The following data was saved to file!")
92          for student in students:
93              print(f"Student {student[0]} {student[1]} is enrolled in {student[2]}")
94          continue
```

Figure 3: This image contains the starter code (lines 81-94), Option 3 from the Menu, that opens, saves new input to the CSV file, and closes it, along with code to display a confirmation message.

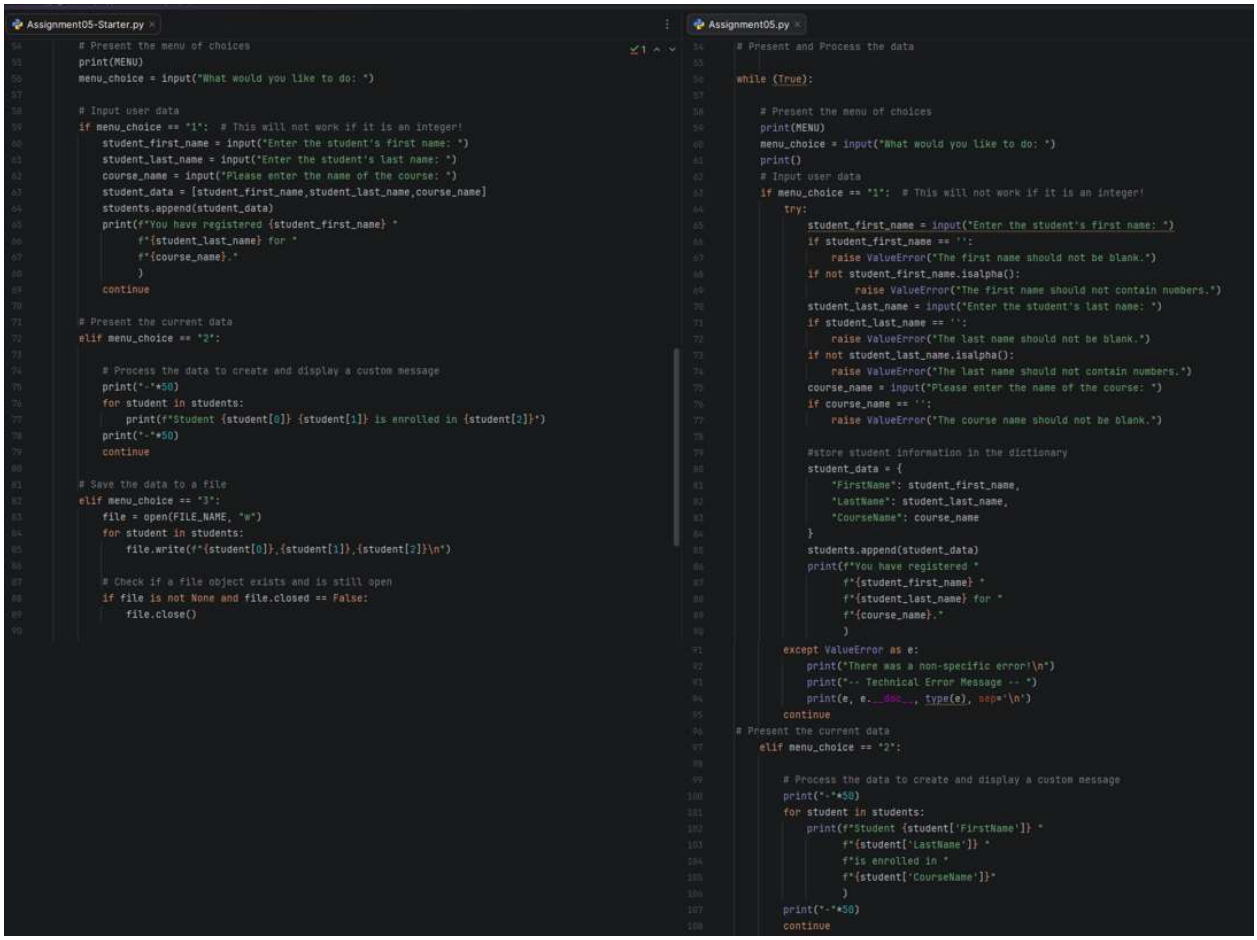
Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05



```
37  # When program starts, read existing registrations
38  try:
39      file = open(FILE_NAME, "r")
40      students = json.load(file)
41
42  except FileNotFoundError as e:
43      print("Text file must exist before running this script!\n")
44      print("-- Technical Error Message -- ")
45      print(e, e.__doc__, type(e), sep='\n')
46  except Exception as e:
47      print("There was a non-specific error!\n")
48      print("-- Technical Error Message -- ")
49      print(e, e.__doc__, type(e), sep='\n')
50  finally:
51      if file is not None and file.closed == False:
52          file.close()
53
```

Figure 4: This image contains the modified code to read an existing JSON using “try”, “except”, and “finally” to create a user-friendly way to handle errors. Please note: the append() function from the original code is now located in the Menu 1 Code Block, at line 85 (Figure 5).

Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05



Assignment05-Starter.py

```
    # Present the menu of choices
    print(MENU)
    menu_choice = input("What would you like to do: ")

    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        student_first_name = input("Enter the student's first name: ")
        student_last_name = input("Enter the student's last name: ")
        course_name = input("Please enter the name of the course: ")
        student_data = [student_first_name,student_last_name,course_name]
        students.append(student_data)
        print(f"You have registered {student_first_name} "
              f"{student_last_name} for "
              f"{course_name}."
              )
        continue

    # Present the current data
    elif menu_choice == "2":

        # Process the data to create and display a custom message
        print("-"*50)
        for student in students:
            print(f"Student {student[0]} {student[1]} is enrolled in {student[2]}")
        print("-"*50)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        file = open(FILE_NAME, "w")
        for student in students:
            file.write(f"{student[0]},{student[1]},{student[2]}\n")

        # Check if a file object exists and is still open
        if file is not None and file.closed == False:
            file.close()
```

Assignment05.py

```
# Present and Process the data

while (True):

    # Present the menu of choices
    print(MENU)
    menu_choice = input("What would you like to do: ")
    print()
    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        try:
            student_first_name = input("Enter the student's first name: ")
            if student_first_name == '':
                raise ValueError("The first name should not be blank.")
            if not student_first_name.isalpha():
                    raise ValueError("The first name should not contain numbers.")
            student_last_name = input("Enter the student's last name: ")
            if student_last_name == '':
                raise ValueError("The last name should not be blank.")
            if not student_last_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
            course_name = input("Please enter the name of the course: ")
            if course_name == '':
                raise ValueError("The course name should not be blank.")

            #store student information in the dictionary
            student_data = {
                "FirstName": student_first_name,
                "LastName": student_last_name,
                "CourseName": course_name
            }
            students.append(student_data)
            print(f"You have registered "
                  f"{student_first_name} "
                  f"{student_last_name} for "
                  f"{course_name}."
                  )
        except ValueError as e:
            print("There was a non-specific error!\n")
            print("-- Technical Error Message -- ")
            print(e, e.__doc__, type(e), sep='\n')
        continue
# Present the current data
    elif menu_choice == "2":

        # Process the data to create and display a custom message
        print("-"*50)
        for student in students:
            print(f"Student {student['FirstName']} "
                  f"{student['LastName']} "
                  f"is enrolled in "
                  f"{student['CourseName']}"
                  )
        print("-"*50)
        continue
```

Figure 5: This image contains a side-by-side comparison of the starter code (left) and modified code (right) for Menu Options 1 & 2. Please note the differences between the CSV operations and list formatting in the original as compared to the JSON operations and dictionary formatting using keys. A while loop was added to Menu Option 1 to allow for error handling of unexpected input, specifically if nothing was entered or user input contained a number, and the formatting was updated from printing a list of lists to printing the dictionaries using keys.

Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05

```
110         # Save the data to a file
111         elif menu_choice == "3":
112             file = None
113
114             try:
115                 file = open(FILE_NAME, "w")
116                 json.dump(students, file)
117
118                 print("The following data was saved to file!")
119                 for student in students:
120                     print(f"Student {student['FirstName']} "
121                           f"{student['LastName']} "
122                           f"is enrolled in "
123                           f"{student['CourseName']}"
124                           )
125             except TypeError as e:
126                 print("Please check that the data is a valid JSON format\n")
127                 print("-- Technical Error Message -- ")
128                 print(e, e.__doc__, type(e), sep='\n')
129             except Exception as e:
130                 print("-- Technical Error Message -- ")
131                 print("Built-In Python error info: ")
132                 print(e, e.__doc__, type(e), sep='\n')
133
134             # Check if a file object exists and is still open
135             finally:
136                 if file is not None and file.closed == False:
137                     file.close()
138             continue
```

Figure 6: This image contains the modified code, Option 3 from the Menu, that opens, saves new input to the JSON (lines 115-116), and closes it (lines 136-137), error handling, and code to display a confirmation message (lines 118-124).

No further changes were made to the starter file; the code for Menu Option 4 stayed the same (Figure 7).

```
96      # Stop the loop
97      elif menu_choice == "4":
98          break  # out of the loop
99      else:
100         print("Please only choose option 1, 2, 3, or 4")
101
102 print("Program Ended")
```

```
139     # Stop the loop
140     elif menu_choice == "4":
141         break  # out of the loop
142     else:
143         print("Please only choose option 1, 2, 3, or 4")
144
145 print("Program Ended")
```

Figure 7: This image contains the Starter code (left) and modified code (right) for menu option 4.

Debugging

During development, I encountered syntax (Figure 8) and indentation errors (Figure 9) when testing my work. Some occurred when adding exception-handling statements, which also

Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05

caused errors in the menu options farther down the program. I corrected the indentation and adjusted the "try" and "except" statements until the code was structured correctly.

```
95      # Present the current data
96          elif menu_choice == "2":
97
98              # Process the data to create and display a custom message
99              print("-"*50)
100             for student in students:
101                 print(f"Student {student['FirstName']} "
102                       f"{student['LastName']} "
103                       f"is enrolled in "
104                       f"{student[CourseName]}"
105                       )
106             print("-"*50)
```

Figure 8: This image contains an error I encountered, where one of the keys was not in single quotes

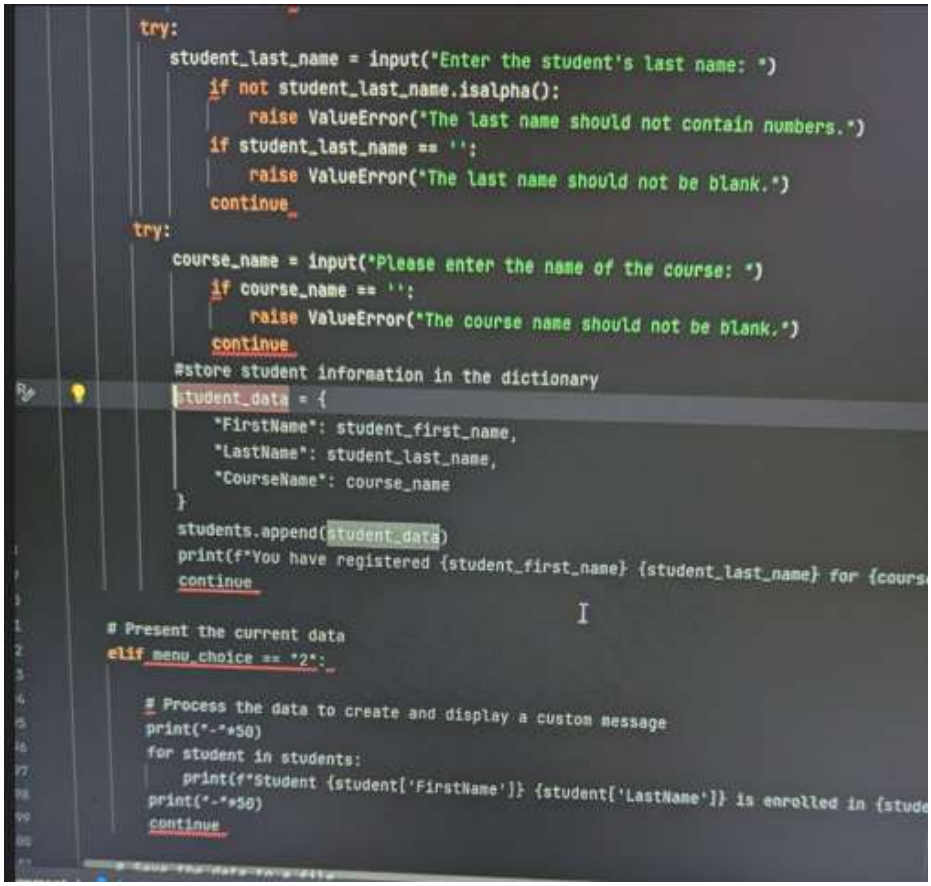


Figure 9: This image contains an indentation error I encountered while attempting to set up Menu Option 1; the code was corrected by nesting all of the user input related code under a single "try" statement as seen in Figure 5.

Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05

Testing the program

After debugging, the completed program was tested in PyCharm and Windows Command Prompt. In PyCharm, I registered a new student, displayed the current registrations, saved the data, and exited the program (Figure 10). The program completed successfully with exit code 0. Next, I tested the program in Windows Command Prompt. I registered Elle Woods for Python 100 using menu option 1. Option 2 displayed the existing registrations along with the new student. Option 3 saved the records, and option 4 ended the program (Figure 11). Finally, I opened "Enrollments.json" in Windows Notepad to verify the saved information. The file contained five student registrations stored as dictionaries in JSON format (Figure 12).These tests confirmed that the program could collect, display, and save multiple registrations and exit normally.

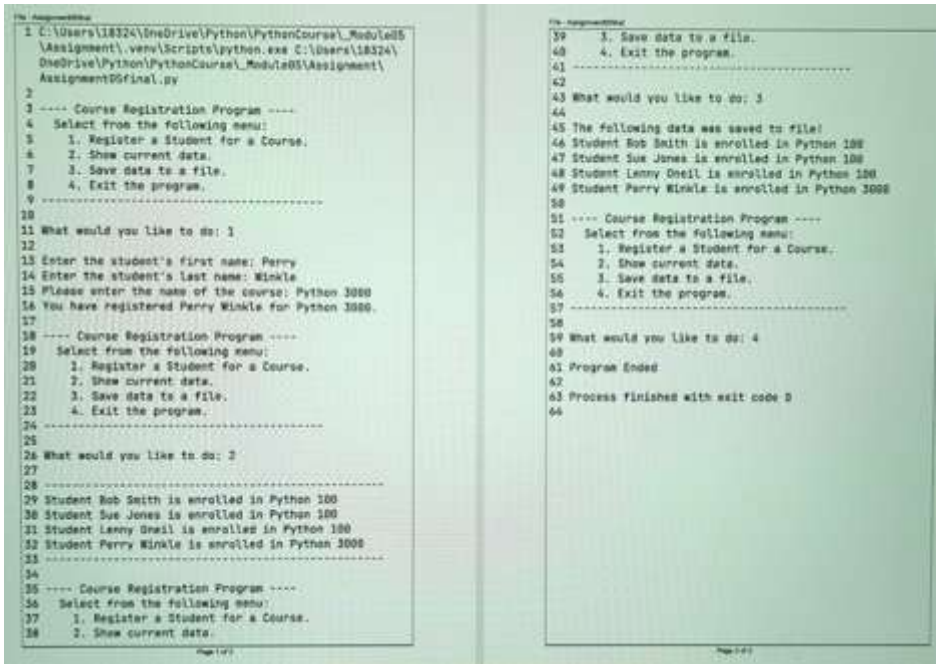


Figure 10: This image shows a PDF printout of testing the program in PyCharm.

Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05

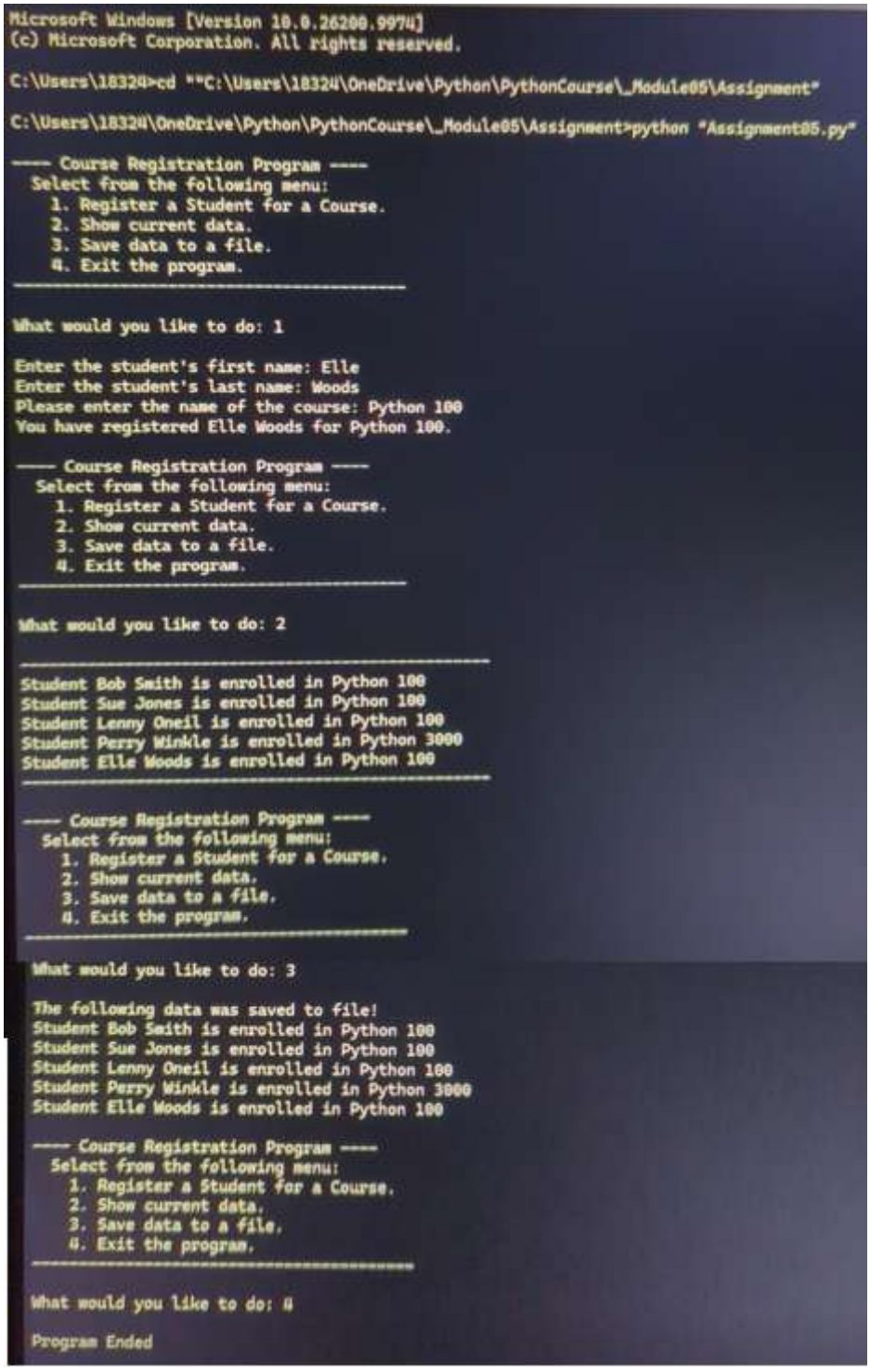


Figure 11: This image shows testing the program Windows Command Prompt.

Moriah Tillman
November 12, 2026
IT FDN 100 GA
Assignment 05

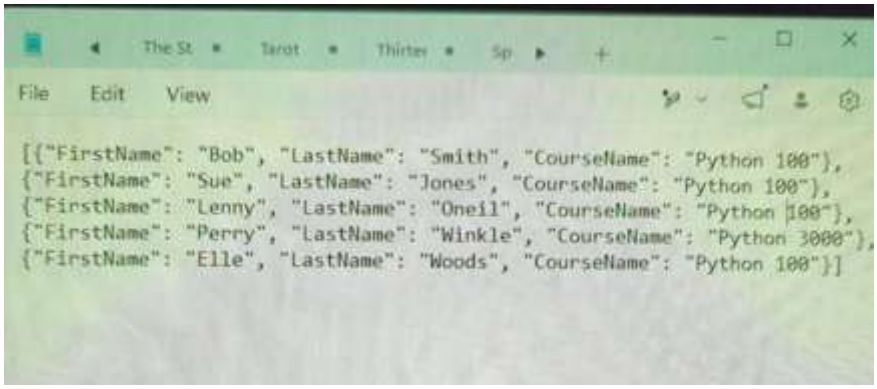


Figure 12: This image shows the saved "Enrollments.json" file displayed in Windows Notepad.

Summary

This assignment demonstrated how to organize information using dictionaries and save structured data using JSON files. I also learned how exception handling can be used to manage errors during program execution. Troubleshooting the code helped me better understand Python's indentation rules and how errors in one section can affect the rest of the program. Testing in PyCharm, Windows Command Prompt, and Notepad helped verify that the program performed the required operations.